No See Code — CTF Writeup

No See Code — CTF Writeup

Platform: Perfect Root (AfricaHackon) **Room:** No See Code **Author:** p3rf3ctr00t **Difficulty:** Easy **Points:** 150 **Category:** Forensics / Steganography — Zero-Width Characters **Result:** Solved (1/1 flag captured)

Description

> Crack and inspect the provided archive. The real message is hidden using zero-width characters.

Flag

`r00t{H1dd3n_1n_Pl41n_S1ght}`

Files Provided

- `no-see-code.zip` (330 bytes) — contains `not_fl4g.txt` (645 bytes), password-protected

Stage 1 — Crack the Archive

`unzip` prompted for a password on `not_fl4g.txt`. Extracted the hash with `zip2john`:

```
$ zip2john no-see-code.zip > zhash.txt
$ cat zhash.txt
ver 2.0 efh 5455 efh 7875 no-see-code.zip/not_fl4g.txt PKZIP Encr: TS_chk, ...

no-see-code.zip/not_fl4g.txt:$pkzip$1*1*2*0*8c*285*ca8032fe*0*46*8*8c*50e5*ea4c0df7f32ac96a154f465f4f5f1
```

Note the header: `PKZIP Encr: TS_chk ... type=8` — classic **ZipCrypto** (legacy PKWARE encryption), supported by John's `PKZIP` format.

Cracked with rockyou:

```
$ john --format=PKZIP zhash.txt --wordlist=/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
secret123        (no-see-code.zip/not_fl4g.txt)
1g 0:00:00:00 DONE
```

Password: `secret123`.

Extracted:

```
$ unzip -P secret123 -o no-see-code.zip
$ ls -la not_fl4g.txt
-rw-r--r-- 1 somasec somasec 645 Nov 28  2025 not_fl4g.txt
```

Stage 2 — Inspect the File

```
$ cat not_fl4g.txt
r00t{naah_not_the_flag}
```

A decoy. The 645-byte file contains far more than 22 visible characters.

```
$ xxd not_fl4g.txt | head -20
00000000: 72e2 808b e280 8b30 e280 8b30 e280 8c74  r......0...0...t
00000010: e280 8c7b e280 8b6e e280 8c61 e280 8d61  ...{...n...a...a
...
```

The bytes `e2 80 8b`, `e2 80 8c`, `e2 80 8d` are UTF-8 for:

- `U+200B` ZERO WIDTH SPACE
- `U+200C` ZERO WIDTH NON-JOINER
- `U+200D` ZERO WIDTH JOINER

Three distinct invisible characters interleaved with the visible text.

Stage 3 — Understand the Structure

Split the file into **runs of consecutive zero-width characters**, keeping their position relative to visible text:

```
Total runs: 22
  [00] visible='r'  zw_count=  2  seq=BB
  [01] visible='0'  zw_count=  1  seq=B
  [02] visible='0'  zw_count=  1  seq=C
  ...
  [20] visible='g'  zw_count=  1  seq=D
  [21] visible='g'  zw_count=185  seq=BBBCBCCDBBBBCBBDBCCBCCCDBBCCCBDBBCCBCCD...
  [22] visible='}'  zw_count=  0
  [23] visible='\n' zw_count=  0
```

Two distinct layers:

1. **Runs 00–20** — one or two zero-width chars each, decorating the visible characters of `r00t{naah_not_the_`. This is the **decoy channel**.
2. **Run 21** — a 185-character run attached to the final visible `g` before the `}`. **This is the real payload.**

Stage 4 — Decode the Real Payload

The 185-char run uses all three zero-width characters, with `U+200D` acting as a **separator**. Splitting on `D`:

```
groups = big.split("D")
24 groups, each 6-7 chars of B/C
```

Mapping `B` (U+200B) → `1`, `C` (U+200C) → `0`, and treating each B/C group as a **binary byte**:

```
run[21] → 't{H1dd3n_1n_Pl41n_S1ght}'
```

That's the tail of the flag. The head `r00t{` was embedded in the single-bit runs 00–20. Once combined and submitted to the platform, the answer confirmed as:

`r00t{H1dd3n_1n_Pl41n_S1ght}`

The Decoding Script

```
#!/usr/bin/env python3
ZW = {"\u200b":"B", "\u200c":"C", "\u200d":"D"}
text = open("not_fl4g.txt", encoding="utf-8").read()

# Split into runs of consecutive ZW characters
runs = []
cur = []
for c in text:
    if c in ZW:
        cur.append(ZW[c])
    else:
        if cur: runs.append("".join(cur))
        cur = []
if cur: runs.append("".join(cur))

# Decode each run: B=1, C=0, D=separator
decoded = []
for run in runs:
    groups = run.split("D")
    s = "".join(chr(int(g.replace("B","1").replace("C","0"), 2))
                for g in groups if g)
    decoded.append(s)

print("".join(decoded))
```

Output (before padding the prefix to 8 bits): t{H1dd3n_1n_Pl41n_S1ght}

What Worked

1. **Recognising the three zero-width codepoints immediately** from the `e2 80 8b / 8c / 8d` byte sequence in `xxd`. This is the standard pattern for `U+200B`, `U+200C`, `U+200D`.
2. **Splitting into runs of consecutive ZW characters, preserving position** relative to visible text. This revealed the two-channel structure (decoys around the visible text + one big hidden payload).
3. **Systematically trying every mapping** (`B=0/C=1`, `B=1/C=0`, ternary, big-int, various group sizes). Only the correct combination (`B=1, C=0, D=separator`, binary bytes) produced printable output.
4. **Not trusting the decoy.** `r00t{naah_not_the_flag}` was visibly suspicious. The real answer was in the 185-char hidden run.

What Failed (and Why)

1. **Naive binary decoding across all zero-widths.** Ignoring the separator meant `D` was treated as a bit, producing garbage. The separator must be stripped first.
2. **Ternary decoding with 5- or 6-trit groups.** The 207 characters decompose as `207 = 3 × 69`, but grouping by trits doesn't yield ASCII because `D` is a separator, not a digit.

3. **Trusting the visible string.** `r00t{naah_not_the_flag}` is deliberately inviting the solver to submit it. It's wrong.

Real-World Impact

Zero-width character steganography is used in real attacks:

- **Prompt injection into LLMs** — hidden instructions sent to AI systems using zero-width chars, invisible to human reviewers.
- **Watermarking / tracking** — per-user invisible tokens embedded in documents to identify leakers (Canary tokens, safeDocs research).
- **Credential smuggling** — leaking secrets through logs or exported text files that appear identical to normal content.
- **Phishing payloads** — hiding malicious commands in files that look empty or benign.
- **Bypass of naive filters** — DLP tools that strip printable content may leave zero-width payloads behind.

Detection: normalize all input to NFKC, strip codepoints in the `U+200B–U+200F`, `U+2060–U+2064`, `U+FEFF` ranges before processing. For logs and forensic artifacts, always `xxd` or hex-inspect — never trust `cat`.

Fixes (Defensive)

- **Normalize Unicode** at every trust boundary. NFKC handles most confusable cases; explicit ZW-stripping handles the rest.
- **Reject or flag** files whose "visible length" differs significantly from their byte length.
- **DLP / egress filters** should scan for zero-width codepoints in outgoing content.
- **When reviewing suspicious text**, always view in a hex editor or with a tool that highlights non-printable codepoints.

Key Takeaways

- `cat` **lies.** A file that looks empty or harmless may be packed with invisible characters carrying significant payload.
- **Three distinct zero-width characters = a signalling alphabet.** The mapping is not always binary — sometimes one is a separator, sometimes ternary, sometimes a run-length marker. Try variants systematically.
- **Structure before decoding.** Splitting into runs of consecutive invisible chars, preserving visible-text position, reveals how the payload is grouped.
- **Decoys are common.** `not_fl4g.txt`, `r00t{naah_not_the_flag}` — the author is telling you not to stop at the first answer.
- **Hex-dump first,** `cat` **second.** For any file that might contain hidden data, always `xxd | head` before trusting the visible content.

Tools Used

- `zip2john`, `john` — cracking the ZipCrypto password
- `xxd` — spot the zero-width codepoints
- `python3` — run-splitter, decoder variants, byte assembly

Commands That Worked

```
# crack
```

```
zip2john no-see-code.zip > zhash.txt
john --format=PKZIP zhash.txt --wordlist=/usr/share/wordlists/rockyou.txt
unzip -P secret123 no-see-code.zip

# inspect
cat not_fl4g.txt       # decoy
xxd not_fl4g.txt       # reveals e2 80 8b/8c/8d
python3 decode.py      # run-splitter + B=1/C=0/D=sep
```